

```
$ mysql -e "EXPLAIN SELECT * FROM users WHERE email = ?"
```

資料庫索引

// 從 $O(n)$ 全表掃描到 $O(\log n)$ 精準定位

B-Tree

平衡樹

EXPLAIN

執行計劃

失效陷阱

函數與通配符

代價

空間換時間

\$ cat index-plan.md

今日路線

01 / 為什麼

全表掃描 vs 索引查詢

02 / 原理

B-Tree 結構與 EXPLAIN 判讀

03 / 設計

哪些欄位該建、複合索引順序

04 / 陷阱

索引失效的六種姿勢

目標：100 萬筆資料的查詢，從 3000ms 壓到 3ms。

PART 01

為什麼需要索引？

在 1000 頁的字典裡找 Apple

沒有索引：逐頁翻

```
1 資料表：  
2  [1] → [2] → [3] → ... → [1000000]  
3  
4 目標：email = 'hua@email.com'  
5 方式：從第 1 筆逐一檢查
```

EXPLAIN

type: **ALL** (全表掃描)
rows: **1000000**

🕒 $O(n)$ – 資料翻倍，時間翻倍

有索引：直接跳

```
1 索引樹：      root  
2              /   \  
3            branch branch  
4              /   \  
5          leaf   leaf → 資料位址
```

EXPLAIN

type: **ref** (索引查詢)
rows: **1**

🕒 $O(\log n)$ – 3~4 次 I/O 到手

資料量越大，差距越恐怖

資料筆數	無索引	有索引	效能提升
1,000	2ms	1ms	2x
10,000	25ms	1ms	25x
100,000	300ms	2ms	150x
1,000,000	3,500ms	3ms	1000x+

$O(n)$ 線性成長

無索引：資料量 $\times 10$ ，時間 $\times 10$

$O(\log n)$ 對數成長

有索引：資料量 $\times 10$ ，時間幾乎不動

小表 (<1000 筆) 效益不明顯；大表 (>10 萬筆) **必須建索引**。

PART 02

B-Tree 原理

為什麼是 3~4 次 I/O ?

B-Tree : 平衡、有序、高扇出



平衡

所有葉節點同深度

有序

鍵值排序，可二分

範圍查詢

葉節點互相連接

高扇出

樹很矮，I/O 很少

索引的本質：空間換時間

```
1  -- 建立索引：多一份「排序過的目錄」
2  CREATE INDEX idx_user_email ON users(email);
3
4  -- 同一個查詢，執行計劃改變
5  SELECT * FROM users WHERE email = 'hua@email.com';
6  -- type: ALL, rows: 1000000 → type: ref, rows: 1
```

索引是獨立的資料結構

存「鍵值 → 資料位址」，按鍵值排序，指標定位

三種常見類型

主索引（PRIMARY KEY 自動建）、一般索引、唯一索引

\$ 驗證方式永遠是 `EXPLAIN` : type 是 `ref` / `range` 用到索引，`ALL` 就是全表掃描。

PART 03

索引設計原則

建對欄位，排對順序

適合建索引

```
1  -- WHERE 常客
2  CREATE INDEX idx_order_status
3      ON orders(status);
4
5  -- JOIN 關聯欄位
6  CREATE INDEX idx_order_user_id
7      ON orders(user_id);
8
9  -- ORDER BY 常客
10 CREATE INDEX idx_order_created
11     ON orders(created_at);
```

不適合建索引

低區分度欄位

gender 只有兩三種值，索引幫不上忙

頻繁更新的欄位

last_login_time 每次登入都要維護索引

很少查詢的欄位

description 建了也沒人用，白付寫入成本

複合索引：順序決定成敗

```
1  -- 慢查詢：100 萬筆全表掃描
2  SELECT * FROM orders
3  WHERE user_id = 123 AND status = 'completed'
4         AND order_date >= '2024-01-01';
5
6  -- 建複合索引：等值在前、範圍在後、選擇性高的優先
7  CREATE INDEX idx_orders_user_status_date
8         ON orders(user_id, status, order_date);
9  -- rows: 1000000 → 120, 快 8000 倍
```

1. user_id

等值、選擇性高

2. status

等值、選擇性低

3. order_date

範圍查詢放最後

PART 04

索引失效陷阱

建了索引 $\neq$ 用得到索引

CHECKPOINT_01

name 欄位有 B-Tree 索引。哪個查詢「用得到」索引？

A

WHERE UPPER(name) = 'ZHANG'

B

WHERE name LIKE '%明'

C

WHERE name LIKE '張%'

六種失效姿勢

```
1  SELECT * FROM users WHERE UPPER(name) = 'ZHANG'; -- ✗ 函數
2  SELECT * FROM users WHERE name = 'Zhang';      -- ✓
3
4  SELECT * FROM users WHERE name LIKE '%明';      -- ✗ 前置通配符
5  SELECT * FROM users WHERE name LIKE '張%';      -- ✓
6
7  SELECT * FROM users WHERE age + 10 > 35;        -- ✗ 欄位運算
8  SELECT * FROM users WHERE age > 25;             -- ✓ 運算移到右邊
```

情況	原因	解法
索引欄位套函數	破壞排序	函數索引 (MySQL 8.0+) 或改條件
前置通配符 <code>%x</code>	有序性無用	後置通配符或全文索引
欄位參與運算	無法預算索引值	運算移到條件右邊
隱式型別轉換 / OR / !=	觸發轉換、難優化	型別一致、改 IN / 範圍

天下沒有免費的索引

GAIN

- ✓ 查詢從 $O(n) \rightarrow O(\log n)$
- ✓ ORDER BY / GROUP BY 加速
- ✓ JOIN 效能大幅改善

COST

- × 額外空間 (約原表 10-30%)
- × INSERT / UPDATE / DELETE 變慢 (要維護樹)
- × 過度索引 = 白付寫入稅

80/20 法則：為 20% 最常用的查詢建索引，不是為每個欄位都建一個。

今日重點

索引 = 目錄

B-Tree 讓查詢從 $O(n)$ 變 $O(\log n)$

EXPLAIN 驗證

type: ALL 是警報，ref / range 才及格

複合索引順序

選擇性高在前、等值在前、範圍在後

失效陷阱

函數、前置通配符、欄位運算

索引是空間換時間 —— 讀多寫少的欄位才值得。

Query optimized.

下一步：進階索引實戰 – 複合索引與 Hash 索引

```
$ EXPLAIN SELECT ... -- type: ref, rows: 1
```

b-tree → explain → composite → 避開失效